

The DXA Dislocation Extraction Pipeline in exaStamp

A Workflow Reference

exaStamp / `src/delaunay`

August 4, 2026

Abstract

This document describes, stage by stage, how exaStamp’s Dislocation Extraction Algorithm (DXA) pipeline works: from a raw MPI-distributed atomic configuration to a set of dislocation lines with Burgers vectors, per-atom core membership, and ParaView-ready visualization output. The implementation follows Stukowski, Bulatov & Arsenlis, “*Automated identification and indexing of dislocations in crystal interfaces*”, Modelling Simul. Mater. Sci. Eng. 20 (2012) 085007 [1], cross-referenced directly against OVITO’s own (non-public) DXA source code where the published paper and its public documentation left the mechanism underspecified. The pipeline lives in `src/delaunay/` and is exercised end to end by the regression tests under `data/regression_new/dxa/`.

Contents

1	Overview	2
2	Local structure classification and lattice clusters	3
2.1	Discrete structure classification	3
2.2	Lattice clusters	4
3	Delaunay tessellation	4
4	Ideal edge vectors: the Crystal Path Finder	4
5	Elastic mapping and tetrahedron classification	5
6	Interface mesh construction	6
7	Burgers circuit sweep	6
7.1	Halfedge structure	6
7.2	Seeding	6
7.3	Sweeping: shrink before expand	7
7.4	Facet ownership and stopping	7
7.5	Segment merging	7
8	Per-atom core marking	8
9	MPI cross-rank line stitching	8
9.1	Raw-fragment redundancy	8
9.2	Cross-rank end matching	9
9.3	Self-closing fragments	9
9.4	Cross-rank N -way junction reconstruction	9
9.5	Cross-rank dislocation-id remap	10

10	Line smoothing and coarsening	10
11	Projecting per-atom fields onto dislocation lines	10
12	Output and visualization	10
13	Known limitations	11

1 Overview

Given a snapshot of atom positions (and, for an MPI run, exaStamp’s own domain decomposition with ghost atoms), the pipeline identifies dislocation lines as one-dimensional topological defects in an otherwise-crystalline lattice, each characterized by:

- a curve in real space (a polyline, possibly a closed loop),
- a Burgers vector $\mathbf{b}$, expressed in the ideal (undeformed) lattice’s reference frame,
- a set of atoms forming its dislocation core.

The published DXA algorithm proceeds in nine steps. Table 1 maps each published step onto the exaStamp operator(s) that implement it, in the order they run in a working `.msp` configuration (see `compute_dxa_elastic_sweep_real_case.msp` for a complete, runnable example).

Paper step	Operator(s)	Role
(i)	<code>compute_dxa_lattice_correspondence</code> <code>compute_dxa_lattice_clusters</code>	Identify crystal-structure classification and local orientation, grouped into contiguous same-orientation clusters.
(ii)	<code>compute_delaunay</code>	Space-filling 3D Delaunay tessellation of owned + ghost atoms.
(iii)	<code>compute_dxa_crystal_path_edges</code>	Assigns an ideal (reference-lattice) vector to every tessellation edge.
(iv)	<code>compute_dxa_elastic_mapping</code>	Classifies each tetrahedron <i>good</i> (elastically compatible with the ideal lattice) or <i>bad</i> (defective).
(v)	<code>compute_dxa_elastic_interface</code>	Builds the 2D interface mesh separating good and bad tetrahedra.
(vi)–(vii)	<code>compute_dxa_circuit_sweep</code>	Grows Burgers circuits by sweeping the interface mesh; a circuit that fails to close trivially encloses a real dislocation core.
(viii)–(ix)	<code>compute_dxa_circuit_sweep</code> , <code>compute_dxa_mpi_stitch_lines</code>	Assembles swept circuits into continuous dislocation lines, merges segments belonging to the same physical dislocation, and stitches lines that cross an MPI rank boundary.

Table 1: Mapping from the published DXA steps to exaStamp operators.

Beyond the nine published steps, this implementation adds three exaStamp-specific stages with no direct DXA-paper analogue:

- `compute_dxa_core_atoms` / `dxa_mark_core_atoms`: marks every atom belonging to a dislocation’s core with that dislocation’s final id.
- `smooth_dxa_dislocation_lines`: coarsens and smooths the raw swept polyline, matching OVITO’s own `linePointInterval/lineSmoothingLevel` post-processing.

- `dxa_project_atom_field`: projects an arbitrary per-atom scalar field onto dislocation line nodes, for coloring the extracted lines by a physical quantity in ParaView.

Figure 1 sketches the full data flow.

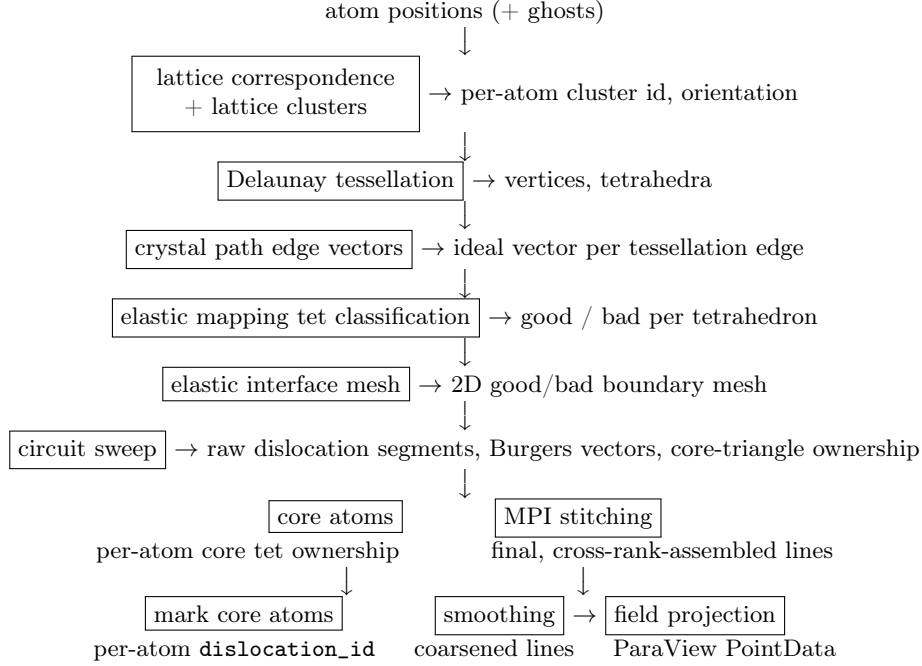


Figure 1: Data flow through the DXA pipeline. The core-atoms and MPI-stitching branches run in parallel and are consumed independently downstream.

2 Local structure classification and lattice clusters

2.1 Discrete structure classification

Every atom is classified against a single, user-specified target lattice (BCC, FCC, or HCP) by `compute_dxa_lattice_correspondence`, a direct port of OVITO’s `StructureAnalysis::determineLocalStructure`. Unlike a continuous orientation-fitting classifier (e.g. Polyhedral Template Matching, PTM [2], used elsewhere in exaStamp but *not* in this pipeline), this is a *discrete graph-topology match*: the atom’s neighbor bond graph (a Common-Neighbor-Analysis-style signature) is compared, permutation by permutation, against the target structure’s fixed reference neighbor template until a bond-topology-exact correspondence is found or every permutation is exhausted. This backtracking search recovers, for a matched atom, a full one-to-one correspondence between the atom’s real neighbors and the ideal lattice’s template neighbor slots (not just a rotation estimate) — which is exactly the input the next two stages need.

Why the target structure must be declared explicitly: an earlier version tested every candidate structure (FCC, HCP, BCC) unconditionally for each atom. Atoms sitting within a few Å of a real junction can, from local strain distortion alone, spuriously satisfy a *different* structure’s own combinatorial test even though the whole system is a single known phase — closing an otherwise-good classification with a handful of false positives near exactly the regions that matter most (dislocation junctions). Restricting the test to only the pipeline’s own declared `target_structure` removes this failure mode entirely and matches OVITO’s own design (a DXA run always targets one declared crystal structure).

2.2 Lattice clusters

`compute_dxa_lattice_clusters` (`dxa_build_lattice_clusters`, a port of `StructureAnalysis::buildClusters`) grows contiguous clusters of atoms that share one consistent crystal orientation: starting from an unvisited classified atom, a breadth-first search crosses a bond to an unassigned same-structure neighbor only if a 3×3 transformation between the two atoms' independently-chosen neighbor-to-template-slot labelings matches one of the structure's known point-group symmetry elements (identity tolerance 10^{-4}). A bond that fails this test is a local grain- or defect-boundary crossing and is simply not traversed — the cluster boundary is the classification result, not a separate step.

Bonds crossing between two *different* already-formed clusters are tested the same way and, if proper-orthogonal, recorded as a `ClusterTransition` edge in a `ClusterGraph`. Each cluster also accumulates a least-squares orientation fit (outer-product accumulation of matched bond vectors) for diagnostic/visualization purposes only.

Determinism across MPI ranks. The BFS seed order matters: the very first bond a cluster's BFS crosses defines an arbitrary reference frame for every subsequent same-cluster atom's own stored neighbor-permutation index. Seeding in raw local-array order makes this reference frame depend on how the domain happens to be decomposed — two ranks' fragments of the very same physical crystal grain can end up with different, only sign-related Burgers vectors downstream. Seeding instead by each atom's persistent, cross-rank-identical `id` makes the reference frame reproducible regardless of rank count or thread scheduling.

3 Delaunay tessellation

`compute_delaunay` builds a 3D Delaunay tessellation of each MPI rank's own owned + ghost atoms using Geogram's `GE0::PeriodicDelaunay3d` [3], constructed in *non-periodic* mode: exaStamp's own ghost atoms are already literal duplicated position copies for both periodic images and cross-rank neighbors, so Geogram never needs its own periodicity machinery (which is designed for one complete point set, not domain-decomposed data).

Tetrahedron ownership. A tetrahedron is kept by a rank only if its centroid falls inside one of that rank's own *owned* (non-ghost) cells. This single rule guarantees the union of every rank's own output has neither gaps nor duplicates (every point in space belongs to exactly one rank's owned region), and simultaneously discards exactly the tetrahedra that would be geometrically unreliable: Delaunay tessellation has no fixed cutoff radius the way a physical pair interaction does, so a tetrahedron near the outer edge of the ghost halo is not guaranteed correct — such tetrahedra only occur far from any owned cell, which the ownership rule already excludes.

An optional `keep_ghost_tets` mode additionally retains tetrahedra centered in *ghost* cells, giving each rank a full local view extending into its own ghost halo (deliberately overlapping the equivalent view from a neighboring rank). This is what later lets a single rank's own dislocation-line sweep (§7) grow a complete circuit even when the real core straddles the owned/ghost seam, rather than being artificially cut off there — the overlap is reconciled afterward by MPI stitching (§9).

4 Ideal edge vectors: the Crystal Path Finder

`compute_dxa_crystal_path_edge_vectors` assigns an *ideal lattice vector* to every Delaunay tessellation edge — the vector that edge *would* have if both its endpoints sat on an undistorted reference lattice. This is what lets later stages measure “how far from ideal” a loop of real edges is.

For two directly-bonded, individually-classified atoms this is immediate: their own stored neighbor-to-template-slot correspondence (§2.1) already gives the answer directly. The harder case is two atoms that are Delaunay-connected but *not* within each other’s native neighbor shell (routine near a defect, where the tessellation can connect atoms across a locally distorted gap). `CrystalPathFinder` [1] resolves this with a graph walk that steps only through atoms belonging to the *same, good crystal cluster* (§2.2) — never through a defective atom — accumulating the template-frame displacement at each hop, then re-expressing the accumulated path vector back into the target edge’s own endpoint’s frame via that cluster’s orientation transformation. Because the walk only ever crosses well-classified territory, it is far more robust near a dislocation core than a scheme relying on the two endpoints’ own direct, possibly-nonexistent neighbor relationship.

Vertex-level cluster propagation. A tessellation vertex is not necessarily a classified atom at all (Delaunay vertices are exactly the atoms, but an atom can be crystallographically unclassified — e.g. sitting inside a defect core). Whether an edge is even eligible for a path search depends on both endpoints already having *some* cluster identity, not necessarily from direct classification: cluster membership is propagated outward from classified atoms across ordinary tessellation-edge adjacency (a breadth-first flood fill), matching OVITO’s own `ElasticMapping::assignVerticesToClusters`. Omitting this propagation step and gating directly on each atom’s own *unpropagated* classification rejects every edge touching *any* unclassified atom outright, well before the path search ever gets a chance to route around it — this single gap was responsible for a $\sim 3\times$ inflation of the downstream interface mesh size in early testing, resolved once the propagation was added.

5 Elastic mapping and tetrahedron classification

`compute_dxa_elastic_mapping_tet_classification` decides, per tetrahedron, whether the four atoms at its corners are locally consistent with the ideal crystal (*good*) or not (*bad*). This is a direct port of OVITO’s `ElasticMapping::isElasticMappingCompatible` and is the single most consequential classification step in the whole pipeline: it is what actually distinguishes a real defect network from ordinary elastic strain.

The test, conceptually. Each of the tetrahedron’s own six edges already carries both a real (measured) displacement and an ideal (reference-lattice) vector from §3. The test asks two related questions:

1. *Burgers-circuit closure.* Do the six ideal edge vectors, walked around the tetrahedron’s own closed circuit, sum to zero — the discrete statement that a Burgers circuit drawn around this tetrahedron in the ideal lattice closes exactly, with no residual (no enclosed dislocation)?
2. *Frank rotation consistency.* Is there one single proper-orthogonal (rotation) transformation mapping every one of the tetrahedron’s own real edge vectors onto its ideal counterpart simultaneously, within tolerance — i.e. is the tetrahedron a genuine, consistently rotated/strained image of the ideal lattice, rather than a locally torn one?

A tetrahedron passes only if both hold; a tetrahedron with even one unresolved edge (§3, no ideal vector could be assigned at all) is automatically *bad* — an unresolved edge is either a real defect signature or a diagnostic gap (see §8 for how these two cases are told apart downstream).

Result character. On an undistorted reference lattice, every tetrahedron passes (100% good). Around a real dislocation network, only a small, spatially tight fraction typically fails (single-digit percent) — this tight, physically localized *bad* region is exactly what the next stage turns into a defect surface.

6 Interface mesh construction

`compute_dxa_elastic_interface_mesh` extracts the 2D manifold boundary separating *good* tetrahedra from *bad* ones: every Delaunay facet (triangle) that has a good tetrahedron on one side and a bad tetrahedron on the other becomes one triangle of the interface mesh. This matches OVITO’s own documented construction exactly.

Because tessellation extends into the ghost halo (§4), a bad tetrahedron near the rank boundary can be genuinely defective *or* merely have an edge whose ideal vector could not be resolved purely because the search ran out of ghost-halo depth (§3), not because of any real defect. Facets touching only this “ghost-halo-limit” kind of bad tetrahedron are excluded from meshing — *but only when the tetrahedron also touches unowned (ghost) territory*; an unresolved-edge tetrahedron deep in owned territory is kept, since that combination *does* occur legitimately right at a real dislocation core, and a blanket exclusion of every unresolved tetrahedron was found to measurably under-count real dislocations.

Each interface-mesh triangle inherits, per edge, the same ideal vector already computed in §3 — no re-derivation is needed, since every interface-mesh edge is a genuine Delaunay tessellation edge. The output `InterfaceMesh` struct (triangles, per-edge ideal vectors, vertex ownership flags) is exactly what the circuit sweep (§7) consumes.

7 Burgers circuit sweep

`compute_dxa_circuit_sweep` is the algorithmic core of the pipeline: it grows Burgers circuits across the interface mesh and turns each one that fails to close trivially into a dislocation line segment with a real Burgers vector. This section follows Stukowski, Bulatov & Arsenlis §§2.4–2.6 [1] directly (the public OVITO documentation summarizes the *effect* — “a hard length limit stops the circuit at a junction” — without describing the actual mechanism; that mechanism only became reproducible after reading the primary paper source, and was cross-checked further against OVITO’s own real (non-public) `DislocationTracer.cpp`).

7.1 Halfedge structure

The interface mesh is treated as a proper halfedge structure: each triangle (v_0, v_1, v_2) owns three directed halfedges $(v_0 \rightarrow v_1), (v_1 \rightarrow v_2), (v_2 \rightarrow v_0)$. On a manifold mesh, exactly one triangle owns any given directed pair — this is what makes “the triangle across this edge” a well-defined, $O(1)$ lookup.

7.2 Seeding

A trial circuit is seeded at a candidate mesh vertex by a bounded-depth breadth-first search (depth $\leq \frac{1}{2}\text{max_circuit_length}$) rooted at that vertex: every non-tree edge the BFS discovers closes a genuinely *local* fundamental cycle (bounded by the search depth by construction). Among all such candidate cycles at that vertex, the shortest one whose own Burgers vector,

$$\mathbf{b} = \oint_C \mathbf{v}_{\text{ideal}}(e) , \quad (1)$$

the discrete line integral of ideal edge vectors around the trial loop C — exceeds a minimum norm threshold (`min_burgers_norm`) is kept as the seed. Vertices already consumed by an earlier line’s own sweep are skipped, and the scan covers every eligible mesh vertex, matching step (viii) of the paper.

7.3 Sweeping: shrink before expand

Once seeded, the circuit advances one elementary move at a time, with an explicit priority: moves that *reduce* the circuit’s length are always preferred over moves that extend it.

- **Shrink move.** If two consecutive circuit halfedges $(a \rightarrow b)$, $(b \rightarrow c)$ are owned by the *same*, still-unclaimed triangle $\{a, b, c\}$, replace them with the single edge $(a \rightarrow c)$ — cutting the corner off that triangle and claiming it for this circuit.
- **Expand move.** Only if no shrink move is available anywhere on the circuit: absorb the third vertex of the (unclaimed) triangle owning one of the circuit’s own halfedges, growing the loop outward by one triangle.

7.4 Facet ownership and stopping

Once a mesh facet has been swept by an advancing circuit, it is marked as belonging to that circuit’s own dislocation segment, and no other circuit may sweep the same triangle again. The sweep genuinely halts — not via an arbitrary length cap — when every candidate move’s own target facet is already claimed, or does not exist (an open, unresolved mesh edge). Four distinct, diagnostic stop reasons are distinguished:

MaxLength the circuit reached `max_circuit_length` (plus `circuit_stretchability` elasticity) without any move available at all — the core is wider than the search is willing to enclose.

SelfClosure the circuit met territory it *itself* already claimed earlier — a genuinely closed dislocation loop.

Junction the circuit met territory claimed by a *different* segment — a real dislocation junction, or (within one MPI rank) two independently seeded segments of the same physical dislocation meeting cleanly (see segment merging below).

OpenEdge the circuit reached an unresolved mesh edge — either a genuine single-rank physical boundary, or (in a multi-rank run) this rank’s own local interface mesh simply not extending past its owned cells; §9 resolves the latter case by stitching.

At each move, the new line vertex is recorded as the circuit’s own current center of mass — literally as specified in the paper — together with the circuit’s own current perimeter (loop size) as a per-point *core size* weight, later used by smoothing (§10).

7.5 Segment merging

A single physical dislocation is frequently discovered as several independently seeded raw segments that happen to sweep into each other’s already-claimed territory — a **Junction** stop does not by itself mean a real branch point, only “not the first segment to reach this facet.” Two segments A and B are merged (their lines spliced together, sharing one continuous id) only when the evidence at the stopping point is unambiguous: OVITO’s own `joinSegments` mechanism scans a stopped circuit’s *entire* boundary via its live edge-ownership map and counts every *distinct* other segment actually touched there. Exactly one distinct neighbor means a clean two-arm pass-through (merge); two or more distinct neighbors is the direct, local signature of a real multi-way junction, and every arm is kept separate. (An earlier version only ever recorded a *single* blocking segment per stop and inferred branching from a coarser, non-local incoming-count heuristic — recording every distinct touched segment directly, as above, is both simpler and strictly more accurate.)

Where a genuine multi-way junction is confirmed *within one rank* (three or more arms with a mutually consistent Frank-rotation relationship, see §9 for the cross-rank analogue), every arm’s own line is left independently terminating at the shared point, connected there via an explicit junction marker rather than merged into one line.

8 Per-atom core marking

`compute_dxa_core_atoms` answers a question the sweep itself never needs to: *which atoms* make up a given dislocation’s core (as opposed to which mesh triangles the circuit swept across). `compute_dxa_circuit_sweep` already records, per interface-mesh triangle, which segment’s own advancing circuit claimed it (§7); this claimed-triangle set is exactly the seed region for a bad-tetrahedron flood fill.

Multi-source BFS. All dislocations’ own core regions grow *simultaneously*, not sequentially: starting from every dislocation’s own claimed-triangle-adjacent bad tetrahedra, a breadth-first search expands through bad-tetrahedron-to-bad-tetrahedron face adjacency, with each tetrahedron claimed by whichever dislocation’s front reaches it first (ties broken by ascending dislocation id, for run-to-run determinism). Growing every dislocation’s front at once — rather than flooding one dislocation to exhaustion before starting the next — is what prevents one dislocation’s core from spuriously absorbing a neighbor’s territory wherever their two bad-tet regions physically touch, e.g. near a junction.

Excluding ghost-halo noise, without excluding real cores. The bad-tetrahedron adjacency graph applies exactly the same exclusion rule as the interface mesh itself (§6): a tetrahedron whose only defect signature is an unresolved edge *and* which also touches unowned (ghost) territory is excluded from the flood; the same combination deep in owned territory is kept, since it legitimately occurs at real cores. Excluding *every* unresolved tetrahedron unconditionally was measured to let the flood leak through the much wider ghost-halo population surrounding the true defect region, producing asymmetric core-atom counts between two physically identical, symmetric dislocations in a controlled test case.

`dxa_mark_core_atoms` then writes a per-atom `dislocation_id` field (default `-1` for any atom not part of a dislocation core), resolving each locally owned tetrahedron’s dislocation id through the cross-rank remap produced by MPI stitching (§9) so that atoms on every rank end up labeled with the same, final, post-stitch dislocation numbering.

9 MPI cross-rank line stitching

Every stage described so far runs entirely within one MPI rank’s own local view. Because a physical dislocation can cross a rank’s domain boundary — and, with `keep_ghost_tets` (§4), a rank’s own local sweep may reconstruct a stretch of that dislocation reaching some distance into its own ghost halo — the raw, per-rank line fragments gathered onto rank 0 by `compute_dxa_mpi_stitch_lines` require a genuine reconciliation pass before they represent one, non-duplicated, correctly connected set of physical dislocations.

9.1 Raw-fragment redundancy

Two ranks’ own independent, ghost-extended views of the same physical dislocation can each reconstruct almost the *entire* line, not merely a short overlap near their shared boundary. Naively stitching two such near-complete, redundant copies end to end (because their two ends happen to share ghost atoms) would double the reported length. A fragment is recognized as a full duplicate of a longer one — and dropped before any stitching is attempted — only if it clears an *arc-span-aware* coverage test: every point of the shorter fragment must lie within tolerance of the longer fragment’s own path (plain coverage), *and* the longer fragment’s own best matches for the shorter fragment’s front and back endpoints must be well separated along its own length (at least half the shorter fragment’s own extent). Coverage alone is insufficient: a genuinely distinct, short fragment that merely shares one exact corner point with a much longer, unrelated line also satisfies plain coverage (its entire short span collapses onto that one shared point) without being

a duplicate at all — the arc-span requirement is what tells the two cases apart.

9.2 Cross-rank end matching

Every open (`OpenEdge`-terminated) fragment end carries the real position and global atom id of every mesh vertex in its own final circuit loop at the moment it stopped. Two ends are matched as the same physical crossing point using two levels of evidence, combined:

- *Exact ghost-atom-id matching*: two ends that share three or more of these atom ids are trusted unconditionally as the same crossing.
- *Position gating at the weakest evidence level*: two ends sharing only the bare minimum (two shared atoms) are additionally required to land within `match_tolerance` of each other after periodic minimum-image wrapping — the minimum-evidence level is precisely where two genuinely *different* nearby crossings were found, in testing, to occasionally share just enough atoms to be confused for one.
- *Fallback*: any still-unmatched end is paired with its nearest (periodic-wrapped) still-unmatched end of Burgers-vector-compatible sign, purely by position, as a last resort.

Matched ends are snapped to their shared, averaged position (or, for an exact match, the real position of the shared atom itself) so the stitched seam has no residual gap.

9.3 Self-closing fragments

A fragment that already traces a dislocation’s *entire* periodic length on its own (both ends overshooting slightly past the true global domain boundary, from ghost-extended growth) is recognized directly: both of its own trimmed ends, after wrapping, land close together. Such a fragment is trimmed to the exact global-domain-boundary crossing on each end (a ray–AABB interpolation against the whole simulation domain, not just this rank’s own owned box) and marked as a closed loop *without* overwriting either endpoint’s own coordinate — unlike an ordinary cross-rank match (which safely joins two previously unrelated fragments), overwriting a self-closing fragment’s own endpoint would break the natural continuity of its own unwrapped coordinate sequence and silently double the reported length.

9.4 Cross-rank N -way junction reconstruction

A genuine three-or-more-arm junction can itself straddle an MPI rank boundary, so the same disambiguation needed within one rank (§7) is needed here too, at the level of whole assembled arms rather than triangles. Shared-atom evidence alone is not sufficient — several unrelated dislocations can legitimately pass close together near the same boundary. The test is **Frank’s rule**: for a correctly, consistently oriented set of N arms meeting at one point, their Burgers vectors must sum to zero,

$$\sum_{i=1}^N s_i \mathbf{b}_i \approx \mathbf{0}, \quad s_i \in \{-1, +1\}. \quad (2)$$

Because an arm’s own stored sign has no fixed relationship to which of its two ends is labeled “front” or “back,” the correct sign assignment cannot be guessed from topology alone; instead every one of the 2^{N-1} sign assignments (arm 0 fixed as reference) is tried, and the junction is accepted if *any* assignment sums within tolerance of zero. If validated, every arm’s own already-pinned endpoint is snapped to one shared, averaged junction position; if no assignment closes the circuit, the arms are left standalone — the group is most likely several distinct dislocations that happen to pass near the same boundary, not one real junction.

9.5 Cross-rank dislocation-id remap

Because per-atom core marking (§8) must happen per rank (a tetrahedron only exists on the rank that computed it) but final dislocation identities are only known here, after stitching, on rank 0, `compute_dxa_mpi_stitch_lines` additionally builds a *local-to-final* dislocation-id remap for every raw fragment — including fragments dropped as redundant, resolved through to whichever surviving fragment they duplicate — and sends each originating rank its own slice of that remap back via a targeted point-to-point message. This is what lets `dxa_mark_core_atoms` (§8) label atoms with the same final dislocation numbering on every rank.

10 Line smoothing and coarsening

`smooth_dxa_dislocation_lines` post-processes the raw, densely-pointed swept polyline into a visually and statistically cleaner curve, matching OVITO’s own `linePointInterval/lineSmoothingLevel` mechanism:

1. **Adaptive coarsening.** Consecutive raw points are merged into groups targeting a fixed target spacing (`target_point_interval`), weighted by each point’s own recorded core size (§7) — a wider circuit (typically near a junction, hence noisier) is merged more aggressively than a narrow, well-defined one. Open-segment endpoints are always pinned exactly (so junction connectivity is never disturbed by coarsening); a closed loop gets a proper wrap-around seam.
2. **Smoothing.** A 2D Taubin (SIGGRAPH 95) alternating λ/μ relaxation runs for `target_smoothing_level` iterations, removing high-frequency zig-zag from the sweep’s own discrete triangle-by-triangle motion without the length-shrinking bias a naive Laplacian smoother would introduce.

On a real, junction-dense dislocation network this typically reduces the raw point count by an order of magnitude while removing a substantial fraction of purely geometric tortuosity (not real physical length); on an already near-straight test line, the same procedure leaves the total length essentially unchanged, confirming it removes genuine zig-zag rather than applying a systematic shortening bias.

11 Projecting per-atom fields onto dislocation lines

`dxa_project_atom_field` lets any other per-atom scalar field already computed elsewhere in the pipeline (a structure-classification id, a stress/strain component, ...) be visualized *on* the extracted dislocation lines, rather than only via a fixed Burgers-vector or dislocation-id coloring. Every rank flattens its own owned atoms’ (position, field value) pairs and gathers them to rank 0 (the same convention used throughout this pipeline for other rank-0-centralized outputs); for each final, post-stitch line node, every gathered atom within a user-specified cutoff (periodic-image aware) is averaged, falling back to the single nearest atom if none fall within cutoff, so that every node always receives a real value. The result is written by `write_dxa_dislocation_lines` as an additional VTK `PointData` array alongside the line geometry itself.

12 Output and visualization

`write_dxa_dislocation_lines` writes the final `DXADislocationLines` as a VTK unstructured-grid piece: one `VTK_POLY_LINE` per line (open segment or closed loop), one `VTK_VERTEX` per within-rank junction and one per reconstructed cross-rank junction, with `burgers_vector` and `dislocation_id` `CellData` and, optionally, a projected-field `PointData` array (§9). A `.pvtu` master file lets ParaView open every rank’s own piece as one dataset.

`write_dxa_ca_file` writes the same result in OVITO's own `.ca` text format, for direct visual/quantitative comparison against real OVITO output.

`write_interface_mesh`, `write_ovito_interface_mesh` write the interface mesh itself (§6), in exaStamp's native `.pvtu` convention and OVITO's own legacy VTK format respectively, for inspecting the good/bad boundary surface directly.

`write_delaunay_vtk` writes the raw tessellation (§4), optionally colored by any named per-particle field, for low-level debugging of the classification/tessellation stages.

13 Known limitations

- **Order-dependent seed-discovery race.** Two real, nearby dislocation arms can compete for the same territory during seeding (§7); whichever is tried first (by raw mesh vertex order) claims it, and the second can fail to be seeded at all rather than erroring. Reversing the scan order changes which arms win but does not remove the race — a genuinely order-independent seeding strategy (e.g. a most-locally-constrained-first priority) would be needed to close this gap; not yet implemented.
- **Coherent-interface handling.** A stacking fault or coherent twin boundary (locally a different but closely related structure embedded in the matrix) is not specially re-classified before circuit sweeping in this implementation, unlike OVITO's own optional `handleCoherentInterfaces` pass.
- **Full-3D domain decomposition at high rank counts.** The cross-rank stitching model (§9) is fundamentally pairwise (each crossing point is assumed touched by exactly two ranks). A fine enough 3D decomposition can put three or more ranks' own ghost views in contact at once near a domain corner; boundary pinning (§9) resolves the resulting position-precision problem, but a real physical junction landing exactly on such a corner remains a harder, only partially explored case.

References

- [1] A. Stukowski, V. V. Bulatov, A. Arsenlis, *Automated identification and indexing of dislocations in crystal interfaces*, Modelling and Simulation in Materials Science and Engineering **20** (2012) 085007.
- [2] P. M. Larsen, S. Schmidt, J. Schiøtz, *Robust structural identification via polyhedral template matching*, Modelling and Simulation in Materials Science and Engineering **24** (2016) 055007.
- [3] B. Lévy et al., *Geogram: a programming library of geometric algorithms*, <https://github.com/BrunoLevy/geogram>.